

DIY Challenge Robot Competition Day Playbook

Power-On Sequence • Pre-Flight Checks • Start-Line Procedure
Live Monitoring • Emergency Procedures • Post-Run Recovery

ROS 2 Humble • Jetson JetPack 5.x

Table of Contents

Section	Topic
1	The Night Before — Pre-Competition Checklist
2	Power-On & Boot Sequence
3	Environment Setup & nvpmode
4	Localization Initialization
5	Start-Line Procedure
6	During-Run Monitoring
7	Post-Run: Bag Recording & Data Off-Load
8	Common Failure Responses
9	Emergency Stop Procedure

1 The Night Before — Pre-Competition Checklist

Complete these tasks the evening before competition day. Do not leave them to the morning — any failure discovered overnight gives time to recover.

1.1 Software & Map Bundle Deploy

Deploy the calibration-and-map bundle to the Jetson:

From your laptop

```
bash scripts/deploy_bundle.sh
```

What it does:

```
# 1. rsync src/challenge_bringup/maps/          → jetson:~/robot_ws/maps/
# 2. rsync src/diy_localization/config/*.yaml    → jetson:~/robot_ws/config/
# 3. rsync scripts/                             → jetson:~/robot_ws/scripts/
# Skips .pcd files >100 MB unless --maps flag passed
```

1.2 Night-Before Checklist

■	deploy_bundle.sh completed without errors
■	static_map.yaml points to correct .pcd files on Jetson
■	Calibration YAMLS have non-placeholder values (no "CALIB:" entries with 0.0)
■	USB drive mounted and writable at /mnt/usb — test: touch /mnt/usb/.test
■	Battery fully charged — check cell voltages, not just SOC leds
■	All ethernet/USB cables connector firm — wiggle test each
■	Hesai IP reachable from Jetson: ping 192.168.1.201
■	STM32 serial port shows up: ls /dev/ttyACM*
■	GPS antenna clear sky view from mounting position (test fix in open area)
■	Run health_check.sh — all PASS

2 Power-On & Boot Sequence

CRITICAL: Power components in order. Wrong order can brownout the Jetson or damage the lidar power supply.

2.1 Power-On Order

1. **Main battery** — connect main LiPo. Check voltage indicator: must be > 22 V (6S).
2. **12 V regulation board** — flip the 12 V rail switch. Hesai QT64 will begin spinning (you may hear the motor start).
3. **Jetson power switch** — hold the power button 1 s. Wait for the green LED steady.
4. **STM32/micro-ROS serial** — should auto-enumerate on /dev/ttyACM0 once Jetson boots.
5. **GPS antenna** — if external power needed, connect now. Expect 30–120 s to first fix.
6. **RealSense D435i** — powered via USB3 from Jetson; auto-enumerates on boot.

2.2 Boot Time Expectations

Component	Expected Time	Verification
Jetson OS	~30 s	Green status LED; ssh jetson@<IP> responds
Hesai QT64 spinning	~10 s after 12V power	Audible rotation hum
GPS first fix (cold start)	60–120 s	ros2 topic echo /gps/fix: status.status = 0
GPS RTK fix	2–5 min after first fix	status.status = 2 (GBAS fix)
micro-ROS agent	5 s after Jetson boot	/dev/ttyACM0 exists; node appears in ros2 node list

3 Environment Setup & nvpmode

3.1 Set Jetson Power Mode

Set maximum performance mode before sourcing ROS:

```
# SSH into the Jetson
ssh jetson@<JETSON_IP>

# Set MAXN mode (all CPU/GPU cores, max frequency)
sudo nvpmode -m 0
sudo jetson_clocks

# Verify
sudo nvpmode -q
# Should print: NV Power Mode: MAXN
```

WARNING: Run `nvpmode` BEFORE starting any ROS nodes. Changing power mode while nodes are running may cause temporary CPU frequency drops that destabilize FAST-LIO2 timing.

3.2 Source the Jetson Profile

In every new terminal on the Jetson:

```
source ~/robot_ws/scripts/env.sh jetson
```

```
# Key variables set by jetson.env:
# DIY_HESAI_IP=192.168.1.201
# DIY_MICRO_ROS_SERIAL=/dev/ttyACM0
# DIY_BAG_PATH=/mnt/usb/bags
# DIY_USE_NAV2=true
# DIY_USE_JOYSTICK=false
# DIY_MUX_MODE=AUTONOMOUS
```

3.3 Run Health Check

Verify all hardware before proceeding:

```
bash ~/robot_ws/scripts/health_check.sh
# Expected: all PASS
# FAIL on hesai/points → check Ethernet cable + IP; ping 192.168.1.201
# FAIL on imu/data → check /dev/ttyACM0; restart micro_ros_agent
# FAIL on TF tree → robot_description node may not be running
```

4 Localization Initialization

Start the localization stack and allow FAST-LIO2 to lock to the prior map before entering the start box. At least 60 seconds of static observation is recommended.

4.1 Start Full Stack (Challenge Master)

Launch everything with `challenge_master.launch.py`:

```
source ~/robot_ws/scripts/env.sh jetson
ros2 launch challenge_bringup challenge_master.launch.py \
  use_nav2:=true \
  use_joystick:=false \
  mux_mode:=AUTONOMOUS
```

Or start localization only for a pre-run check:

```
ros2 launch diy_localization localization.launch.py \
  mode:=runtime use_gps:=true
```

4.2 Localization Lock Checklist

■	FAST-LIO2 prints "IMU Initializing" then "LiDAR point cloud registration" — wait for stability
■	/odometry/filtered publishing at 50 Hz — check: <code>ros2 topic hz /odometry/filtered</code>
■	Covariance in /odometry/filtered diagonal entries < 0.05 (stable lock)
■	TF tree complete: <code>ros2 run tf2_ros tf2_echo map base_link</code> returns pose (not error)
■	map→odom TF is being published (EKF2 active)
■	GPS fix acquired (status ≥ 0): <code>ros2 topic echo /gps/fix --once</code>
■	/odometry/gps and /odometry/filtered agree within 1 m (pre-start sanity)

WARNING: Do not drive to the start line until localization is locked (covariance stable). Starting with diverged pose estimate causes Nav2 to plan from wrong position.

5 Start-Line Procedure

5.1 Drive to Start Line (Manual Mode)

Switch to joystick/teleop mode to position at the start line:

```
# Enable joystick from a second terminal
ros2 topic pub /cmd_vel_mux/mode std_msgs/msg/String "data: JOYSTICK" --once
```

```
# Or restart challenge_master with joystick enabled:
ros2 launch challenge_bringup challenge_master.launch.py \
  use_nav2:=true \
  use_joystick:=true \
  mux_mode:=JOYSTICK
```

NOTE: The cmd_vel_mux arbitrates between JOYSTICK and AUTONOMOUS modes. In JOYSTICK mode, joystick commands pass through even with Nav2 running. Estop signal from STM32 overrides both modes.

5.2 Start-Line Lock-In Checklist

■	Robot physically positioned at start line, facing the course direction
■	Localization pose in RViz matches physical robot position (< 0.3 m error)
■	No obstacles within 1 m of robot (collision monitor will abort navigation otherwise)
■	Nav2 goal server online: <code>ros2 action list grep NavigateToPose</code>
■	Confirm mux_mode = AUTONOMOUS before sending first goal
■	Record bag started (see §7)
■	Announce "ready" to competition official

5.3 Send the First Navigation Goal

Send goal via ros2 action CLI or a pre-defined task script:

```
# Example: send goal to GPS waypoint (lat/lon via navigate_through_poses)
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: map}, pose: {position: {x: 10.0, y: 5.0, z: 0.0},
  orientation: {w: 1.0}}}}"
```

```
# Or use the pre-defined waypoint follower if configured:
ros2 topic pub /waypoint_list nav2_msgs/msg/WaypointArray ...
```

6 During-Run Monitoring

One team member watches RViz and the terminal output during the run. Know the override commands before the run starts.

6.1 What to Watch in RViz

- **Robot footprint marker** — should track physical robot position smoothly.
- **Planned path** (blue line) — should be a sensible route, not looping.
- **Local costmap** — red zone around the robot reflects real-time obstacles.
- **Global costmap** — should not have phantom obstacles from lidar noise.
- **Nav2 status panel** — watch for "Recovery" actions (spinning, backing up).

6.2 Key Terminal Monitoring Commands

Monitor from a second terminal (run simultaneously):

```
# Nav2 planner/controller status
ros2 topic echo /navigate_to_pose/_action/status
```

```
# Current velocity commands
ros2 topic echo /cmd_vel --once
```

```
# CPU and memory on Jetson
jtop # install: sudo pip3 install jetson-stats
```

```
# Estop status
ros2 topic echo /estop_active --once
```

```
# Lidar health
ros2 topic hz /hesai/points # must stay at ~20 Hz
```

6.3 Speed Limits

Parameter	Value	Where Set
Max forward speed (vx_max)	0.6 m/s	nav2_params.yaml — FollowPath.MotionModel
Max reverse speed (vx_min)	-0.1 m/s	nav2_params.yaml
Max rotation speed (wz_max)	1.2 rad/s	nav2_params.yaml
Goal tolerance x/y	0.20 m	nav2_params.yaml — goal_checker
Goal tolerance yaw	0.20 rad	nav2_params.yaml — goal_checker
Progress timeout	8 s	nav2_params.yaml — progress_checker

7 Post-Run: Bag Recording & Data Off-Load

7.1 Start Recording Before the Run

Start bag recording before sending the first goal:

```
source scripts/env.sh jetson
bash scripts/record_bag.sh
# Saves to: $DIY_BAG_PATH = /mnt/usb/bags/bag_YYYYMMDD_HHMMSS/
# Topics recorded: /hesai/points /imu/data /gps/fix
# /odometry/filtered /odometry/global /cmd_vel /cmd_vel_safe
# /tf /tf_static /estop_active /navigate_to_pose/_action/*
```

7.2 Stop Recording and Verify

After the run, Ctrl+C the record terminal, then verify:

```
# Check bag is not empty
ros2 bag info /mnt/usb/bags/bag_<TIMESTAMP>/
# Should show: duration > 0s, all topics have messages

# Quick replay check (no network required)
bash scripts/replay_bag.sh /mnt/usb/bags/bag_<TIMESTAMP>/
```

7.3 Data Off-Load

Copy bags to laptop for post-processing:

```
# From laptop
rsync -av jetson@<JETSON_IP>:/mnt/usb/bags/ ~/bags/

# Or eject the USB drive and copy directly
umount /mnt/usb      # on Jetson
# physically remove drive, insert in laptop
```

NOTE: Note the run timestamp and any anomalies observed during the run. This makes post-run analysis faster when reviewing LIO-SAM / EKF logs.

8 Common Failure Responses

Failures are most likely to occur at start-line initialization or after a hard turn. Know these responses before competition begins.

Failure	Symptom	Response
Nav2 stuck in recovery	Robot spins in place or backs up repeatedly	Check costmap for phantom obstacle; ros2 topic pub /initialpose — relocalize
FAST-LIO2 diverged	/odometry/filtered pose jumps >2 m; map display goes wrong	kill fastlio_mapping, restart localization.launch.py, confirm lidar is clean
Hesai lidar goes silent	/hesai/points drops to 0 Hz; health_check FAIL	Ping 192.168.1.201; reconnect Ethernet; power-cycle lidar 12V rail
GPS fix lost	/gps/fix status=-1; /odometry/gps stops	Switch use_gps:=false, restart EKF2; robot uses lidar-only localization
STM32 / micro-ROS crash	/cmd_vel_safe not reaching motors; /estop_active stuck true	Reconnect USB; ros2 run micro_ros_agent ... serial; reset robot estop
EKF diverges (covariance bloom)	/odometry/filtered covariance diagonal > 1.0	Restart ekf_node; feed initial pose with ros2 topic pub /initialpose
Nav2 refuses to start	NavigateToPose action server not found	Check map_server has valid map YAML; check bt_navigator logs

9 Emergency Stop Procedure

CRITICAL: Emergency stop cuts motor output immediately. The robot will coast to a stop with no braking. Ensure the area ahead of the robot is clear before resetting.

9.1 Hardware Estop

- Press the physical red estop button on the robot chassis.
- STM32 firmware publishes `/estop_active = true`.
- `cmd_vel_mux` gates all velocity commands — motors stop.
- To resume: twist the estop button clockwise to release, then send `/estop_reset` service call or press the estop release button.

9.2 Software Estop (Remote)

Force estop via topic from any terminal:

```
# Immediate software stop
ros2 topic pub /estop_active std_msgs/msg/Bool "data: true" --once

# Resume when safe
ros2 topic pub /estop_active std_msgs/msg/Bool "data: false" --once

# Cancel all Nav2 goals without estop
ros2 action cancel /navigate_to_pose
```

9.3 Full Shutdown Sequence

1. Press hardware estop button.
2. Kill all ROS nodes: `pkill -f ros2` on Jetson.
3. Stop bag recording if running: `Ctrl+C` on record terminal.
4. Shut down Jetson: `sudo shutdown now`.
5. Switch off 12 V rail (lidar power).
6. Disconnect main battery.
7. Do not reconnect battery within 30 s.

End of Competition Day Playbook. For localization bringup detail see the [Mapping & Localization Workflow Guide](#). For Nav2 tuning see the [Nav2 Tuning Guide](#).